OS – Unit 4 Orange Problem

Name : Atul Kandiyil
SRN : PES2UG24CS920

1. Phase 1: -

1.A) output for ./test_objects

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

1.B) output for find .pes/objects -type f

```
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

2. Phase 2: -

2.A) output for ./test_tree

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

2.B) output for xxd .pes/objects <path-to-object>

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ find .pes/objects -type f
.pes/objects/5c/5e3192452601c78cd146924078115a9c3dfa6d8b2274f906e1d1a182a10bfe
.pes/objects/a7/a03e0ccc2432217e0571e0f0e45d5b638fbca0435e3192a26d552f550e4541
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee2
9543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

3. Phase 3: -

3.A) output for ./pes init, ./pes add file1.txt file2.txt and ./pes status

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     demo.txt
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  tree.h
  untracked:  index.h
  untracked:  index.c
  untracked:  tree.c
  untracked:  bye.txt
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  test_tree
  untracked:  pes.h
  untracked:  commit.c
  untracked:  pes.c
```

3.B) output for cat .pes/index

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776797598 6 demo.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776798110 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776798110 6 file2.txt
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

4. Phase 4: -

4.A) output of ./pes log

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 828bf80cf36d... Initial commit
Committed: ea01190764a0... Add world
Committed: 0521cf9b67b3... Add farewell
commit 0521cf9b67b348dfee5879233f47b8ba951de9c7a0a232a1bd10dd7093e8f775
Author: PES User <pes@localhost>
Date:   1776798829

    Add farewell

commit ea01190764a0b1dcd9bbddb643dde5e4f124b16150d5154e97a1f5fcb7436f36
Author: PES User <pes@localhost>
Date:   1776798829

    Add world

commit 828bf80cf36d8f9b49969d081568489eb1dbc051f503288d3641b524dfa5edfd
Author: PES User <pes@localhost>
Date:   1776798829

    Initial commit
```

```
    Initial commit

commit 5c5e3192452601c78cd146924078115a9c3dfa6d8b2274f906e1d1a182a10bfe
Author: PES User <pes@localhost>
Date:   1776797599

    demo

enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

4.B) output of find .pes -type f | sort

```
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/21cf9b67b348dfee5879233f47b8ba951de9c7a0a232a1bd10dd7093e8f775
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/844d5725639d180174d4ebf0d42b3a9625bfb38c72da95ce51e170d229e653
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/44/e4808eb7241d6b4c877b82886dc46771eceb8c017c84851221404a8462ca67
.pes/objects/5c/5e3192452601c78cd146924078115a9c3dfa6d8b2274f906e1d1a182a10bfe
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/82/8bf80cf36d8f9b49969d081568489eb1dbc051f503288d3641b524dfa5edfd
.pes/objects/82/da72c3b16a05c2961723864eb8d7693338d7438ecf1db3ff2fc4294e691194
.pes/objects/a7/a03e0ccc2432217e0571e0f0e45d5b638fbca0435e3192a26d552f550e4541
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/01190764a0b1dcd9bbddb643dde5e4f124b16150d5154e97a1f5fcb7436f36
.pes/refs/heads/main
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

4.C) output of cat .pes/refs.head/main and cat .pes/HEAD

```
PES2UG24CS920-pes-vcs — bash
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ cat .pes/refs/heads/main
0521cf9b67b348dfee5879233f47b8ba951de9c7a0a232a1bd10dd7093e8f775
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

5. make test-integration output: -

```
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 687520082c0b... Initial commit

Log after first commit:
commit 687520082c0b093ba5f3d25a8e1405479a424879996362704084232a2b99d00c
Author: PES User <pes@localhost>
Date:   1776799462

    Initial commit
```

```
--- Second Commit ---
Committed: 94eb2e985132... Update file.txt

--- Third Commit ---
Committed: e3853e6568db... Add farewell

--- Full History ---
commit e3853e6568dbeb08024f4dfc27acdec2f1d4c517fffcf6e4e7300904f0fa1e16
Author: PES User <pes@localhost>
Date:   1776799462

    Add farewell

commit 94eb2e9851320518c26b296f0573e728aba32d94983883e54e5d0762cdaf8d39
Author: PES User <pes@localhost>
Date:   1776799462

    Update file.txt

commit 687520082c0b093ba5f3d25a8e1405479a4248799963627040842 32a2b99d00c
Author: PES User <pes@localhost>
Date:   1776799462

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/68/7520082c0b093ba5f3d25a8e1405479a4248799963627040842 32a2b99d00c
.pes/objects/94/eb2e9851320518c26b296f0573e728aba32d94983883e54e5d0762cdaf8d39
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e3/853e6568dbeb08024f4dfc27acdec2f1d4c517fffcf6e4e7300904f0fa1e16

=== All integration tests completed ===
enforcerxii@AtulKandiyil-22:~/Documents/PES2UG24CS920-pes-vcs$
```

6. Phase 5 & 6: -

Q5.1
To implement pes checkout <branch>, the system should switch HEAD so it points to that branch, read the latest commit of that branch, and then update the files in the project folder to match that commit. It should also update the index so staging matches the checked-out state.
This is complex because checkout must carefully handle modified files, deleted files, and new files without losing user work.

Q5.2
Before switching branches, the system should check if the user has changed any tracked file without committing. If such a file is also different in the target branch, checkout should stop and show an error.

This prevents overwriting local work accidentally.

Q5.3
In detached HEAD state, commits can still be made, but they are not attached to any branch name. So later, if the user switches branches, those commits can be hard to find.
To recover them, the user should create a new branch pointing to that commit hash.

Q6.1
Garbage collection should start from all branch tips and follow commit history and trees to mark all objects that are still in use. Any object not reached this way is unreachable and can be deleted.
A hash set is a good way to keep track of visited objects efficiently.
For a repository with 100,000 commits and 50 branches, the system may need to

scan a very large number of objects, roughly all objects reachable from those branch tips.

Q6.2
Running garbage collection while a commit is happening is risky because commit creation happens in steps. GC might think a newly written object is unused and delete it before the branch pointer is updated.
Git avoids this with locking, careful ordering, and by not immediately deleting very recent unreachable objects.

7. Code files: -

a) object.c

```
// object.c — Content-addressable object
store
//
// Every piece of data (file contents,
directory listings, commits) is stored
// as an "object" named by its SHA-256
hash. Objects are stored under
// .pes/objects/XX/YYYYYY... where XX
is the first two hex characters of the
// hash (directory sharding).
//
// PROVIDED functions: compute_hash,
object_path, object_exists, hash_to_hex,
hex_to_hash
// TODO functions:     object_write,
object_read

#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
```

```
#include <fcntl.h>
#include <unistd.h>
#include <openssl/evp.h>

// ——— PROVIDED ————————————————————————————————————————————————————————————————————————————————

void hash_to_hex(const ObjectID *id,
char *hex_out) {
    for (int i = 0; i < HASH_SIZE; i++) {
        sprintf(hex_out + i * 2, "%02x", id-
>hash[i]);
    }
    hex_out[HASH_HEX_SIZE] = '\0';
}

int hex_to_hash(const char *hex, ObjectID
*id_out) {
    if (strlen(hex) < HASH_HEX_SIZE)
return -1;
```

```
    for (int i = 0; i < HASH_SIZE; i++) {
        unsigned int byte;
        if (sscanf(hex + i * 2, "%2x",
&byte) != 1) return -1;
        id_out->hash[i] = (uint8_t)byte;
    }
    return 0;
}

void compute_hash(const void *data,
size_t len, ObjectID *id_out) {
    unsigned int hash_len;
    EVP_MD_CTX *ctx =
EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha256(),
NULL);
    EVP_DigestUpdate(ctx, data, len);
    EVP_DigestFinal_ex(ctx, id_out->hash,
&hash_len);
    EVP_MD_CTX_free(ctx);
}
```

```
// Get the filesystem path where an object
should be stored.
// Format:
.pes/objects/XX/YYYYYYYY...
// The first 2 hex chars form the shard
directory; the rest is the filename.
void object_path(const ObjectID *id, char
*path_out, size_t path_size) {
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(id, hex);
    snprintf(path_out, path_size,
"%s/%.2s/%s", OBJECTS_DIR, hex, hex
+ 2);
}

int object_exists(const ObjectID *id) {
    char path[512];
    object_path(id, path, sizeof(path));
    return access(path, F_OK) == 0;
}
```

```
// ─── TODO: Implement these ──────────────
```

```
    const char *type_str = (type ==
OBJ_BLOB) ? "blob" : (type ==
OBJ_TREE ? "tree" : "commit");
    char header[64];
    int header_len = sprintf(header, "%s
%zu", type_str, len) + 1;

    // 2. Combine header and data to
compute hash
    size_t total_len = header_len + len;
    uint8_t *full_data = malloc(total_len);
    memcpy(full_data, header, header_len);
    memcpy(full_data + header_len, data,
len);
    compute_hash(full_data, total_len,
id_out);

    // 3. Determine path:
.pes/objects/XX/YYYY...
        char path[512];
        object_path(id_out, path,
sizeof(path));
```

```
        // Create directory (sharding)
        char dir[512];
        strncpy(dir, path, 15); // Extracts
".pes/objects/XX"
        dir[15] = '\0';
        mkdir(dir, 0755);

    // 4. Atomic Write: Write to .tmp then
rename
    char tmp_path[520];
    sprintf(tmp_path, "%s.tmp", path);
    FILE *f = fopen(tmp_path, "wb");
    if (!f) { free(full_data); return -1; }

    fwrite(full_data, 1, total_len, f);
    fclose(f);
    rename(tmp_path, path);

    free(full_data);
    return 0;
```

```
}

int object_read(const ObjectID *id,
ObjectType *type_out, void **data_out,
size_t *len_out) {
    char path[512];
    object_path(id, path, sizeof(path));

    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    // Get file size
    fseek(f, 0, SEEK_END);
    size_t file_size = ftell(f);
    rewind(f);

    uint8_t *full_buf = malloc(file_size ?
file_size : 1);
    if (!full_buf) {
        fclose(f);
        return -1;
    }
```

```
    if (fread(full_buf, 1, file_size, f) !=
file_size) {
        fclose(f);
        free(full_buf);
        return -1;
    }
    fclose(f);

    // 1. Integrity Check
    ObjectID actual_id;
    compute_hash(full_buf, file_size,
&actual_id);
    if (memcmp(id->hash, actual_id.hash,
HASH_SIZE) != 0) {
        free(full_buf);
        return -1;
    }

    // 2. Parse Header
    char *header = (char *)full_buf;
    char *data_start = memchr(header, '\0',
file_size);
```

```
    if (!data_start) { free(full_buf); return -
1; }
    data_start++; // Move past \0

    if (strncmp(header, "blob", 4) == 0)
*type_out = OBJ_BLOB;
    else if (strncmp(header, "tree", 4) == 0)
*type_out = OBJ_TREE;
    else *type_out = OBJ_COMMIT;

    // 3. Extract Data
    *len_out = file_size - (size_t)(data_start
- (char *)full_buf);
    *data_out = malloc(*len_out + 1);
    if (!*data_out) {
        free(full_buf);
        return -1;
    }
    memcpy(*data_out, data_start,
*len_out);
    ((char *)*data_out)[*len_out] = '\0';
```

```
    free(full_buf);
    return 0;

    //end of phase 1
}
```

b) tree.c

```
// tree.c — Tree object serialization and
construction
//
// PROVIDED functions: get_file_mode,
tree_parse, tree_serialize
// TODO functions:     tree_from_index
//
// Binary tree format (per entry,
concatenated with no separators):
//   "<mode-as-ascii-octal> <name>\0<32-
byte-binary-hash>"
//
```

```
// Example single entry (conceptual):
//   "100644 hello.txt\0" followed by 32
raw bytes of SHA-256

#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include "index.h"

// ─── Mode Constants
──────────────────────────────────────────────────────────────────────────────

#define MODE_FILE      0100644
#define MODE_EXEC      0100755
#define MODE_DIR       0040000
```

```
// Forward declaration (implemented in
object.c)
int object_write(ObjectType type, const
void *data, size_t len, ObjectID *id_out);

// ——— PROVIDED
———————————————————————————————————————————————————————————————

// Determine the object mode for a
filesystem path.
uint32_t get_file_mode(const char *path)
{
    struct stat st;
    if (lstat(path, &st) != 0) return 0;

    if (S_ISDIR(st.st_mode))  return
MODE_DIR;
    if (st.st_mode & S_IXUSR) return
MODE_EXEC;
    return MODE_FILE;
```

```
}

// Parse binary tree data into a Tree struct
safely.
// Returns 0 on success, -1 on parse error.
int tree_parse(const void *data, size_t len,
Tree *tree_out) {
    tree_out->count = 0;
    const uint8_t *ptr = (const uint8_t
*)data;
    const uint8_t *end = ptr + len;

    while (ptr < end && tree_out->count <
MAX_TREE_ENTRIES) {
        TreeEntry *entry = &tree_out-
>entries[tree_out->count];

        // 1. Safely find the space character
for the mode
        const uint8_t *space = memchr(ptr, '
', end - ptr);
```

```
	if (!space) return -1; // Malformed
data

	// Parse mode into an isolated buffer
	char mode_str[16] = {0};
	size_t mode_len = space - ptr;
	if (mode_len >= sizeof(mode_str))
return -1;
	memcpy(mode_str, ptr, mode_len);
	entry->mode = strtol(mode_str,
NULL, 8);

	ptr = space + 1; // Skip space

	// 2. Safely find the null terminator
for the name
	const uint8_t *null_byte =
memchr(ptr, '\0', end - ptr);
	if (!null_byte) return -1; // Malformed
data

	size_t name_len = null_byte - ptr;
```

```
        if (name_len >= sizeof(entry-
>name)) return -1;
        memcpy(entry->name, ptr,
name_len);
        entry->name[name_len] = '\0'; //
Ensure null-terminated

        ptr = null_byte + 1; // Skip null byte

        // 3. Read the 32-byte binary hash
        if (ptr + HASH_SIZE > end) return -
1;
        memcpy(entry->hash.hash, ptr,
HASH_SIZE);
        ptr += HASH_SIZE;

        tree_out->count++;
    }
    return 0;
}
```

```
// Helper for qsort to ensure consistent tree
hashing
static int compare_tree_entries(const void
*a, const void *b) {
    return strcmp(((const TreeEntry *)a)-
>name, ((const TreeEntry *)b)->name);
}

// Serialize a Tree struct into binary format
for storage.
// Caller must free(*data_out).
// Returns 0 on success, -1 on error.
int tree_serialize(const Tree *tree, void
**data_out, size_t *len_out) {
    // Estimate max size: (6 bytes mode + 1
byte space + 256 bytes name + 1 byte null
+ 32 bytes hash) per entry
    size_t max_size = tree->count * 296;
    uint8_t *buffer = malloc(max_size);
    if (!buffer) return -1;
```

```
    // Create a mutable copy to sort entries
(Git requirement)
    Tree sorted_tree = *tree;
    qsort(sorted_tree.entries,
sorted_tree.count, sizeof(TreeEntry),
compare_tree_entries);

    size_t offset = 0;
    for (int i = 0; i < sorted_tree.count; i++)
{
        const TreeEntry *entry =
&sorted_tree.entries[i];

        // Write mode and name (%o writes
octal correctly for Git standards)
        int written = sprintf((char *)buffer +
offset, "%o %s", entry->mode, entry-
>name);
        offset += written + 1; // +1 to step
over the null terminator written by sprintf

        // Write binary hash
```

```
        memcpy(buffer + offset, entry->hash.hash, HASH_SIZE);
        offset += HASH_SIZE;
    }

    *data_out = buffer;
    *len_out = offset;
    return 0;
}

// ——— TODO: Implement these ————————————————————————————————————————————————————

// Build a tree hierarchy from the current index and write all tree
// objects to the object store.
//
// HINTS - Useful functions and concepts for this phase:
```

// - index_load : load the staged files into memory
// - strchr : find the first '/' in a path to separate directories from files
// - strncmp : compare prefixes to group files belonging to the same subdirectory
// - Recursion : you will likely want to create a recursive helper function
// (e.g., `write_tree_level(entries, count, depth)`) to handle nested dirs.
// - tree_serialize : convert your populated Tree struct into a binary buffer
// - object_write : save that binary buffer to the store as OBJ_TREE
//
// Returns 0 on success, -1 on error.
//
//

```
static int write_tree_recursive(IndexEntry
*entries, int count, int depth, ObjectID
*id_out) {
    Tree tree;
    tree.count = 0;

    for (int i = 0; i < count; ) {
        const char *current = entries[i].path;
        const char *segment = current;

        for (int d = 0; d < depth; d++) {
            const char *slash = strchr(segment,
'/');
            if (!slash) {
                segment = NULL;
                break;
            }
            segment = slash + 1;
        }

        if (!segment || segment[0] == '\0')
return -1;
```

```
        const char *slash = strchr(segment,
'/');

        if (!slash) {
            if (tree.count >=
MAX_TREE_ENTRIES) return -1;
            if (strlen(segment) >=
sizeof(tree.entries[0].name)) return -1;
            TreeEntry *entry =
&tree.entries[tree.count++];
            entry->mode = entries[i].mode;
            entry->hash = entries[i].hash;
            snprintf(entry->name, sizeof(entry-
>name), "%s", segment);
            i++;
            continue;
        }

        size_t dir_len = (size_t)(slash -
segment);
```

```
        if (dir_len == 0 || dir_len >=
sizeof(tree.entries[0].name)) return -1;

        char dirname[256];
        memcpy(dirname, segment, dir_len);
        dirname[dir_len] = '\0';

        int start = i;
        int end = i;
        while (end < count) {
            const char *candidate =
entries[end].path;
            const char *candidate_segment =
candidate;

            for (int d = 0; d < depth; d++) {
                const char *candidate_slash =
strchr(candidate_segment, '/');
                if (!candidate_slash) {
                    candidate_segment = NULL;
                    break;
                }
```

```
                candidate_segment =
candidate_slash + 1;
            }

            if (!candidate_segment) break;
            if (strncmp(candidate_segment,
dirname, dir_len) != 0) break;
            if (candidate_segment[dir_len] !=
'/') break;

            end++;
        }

        ObjectID child_id;
        if (write_tree_recursive(entries +
start, end - start, depth + 1, &child_id) !=
0) return -1;

        if (tree.count >=
MAX_TREE_ENTRIES) return -1;
        TreeEntry *entry =
&tree.entries[tree.count++];
```

```
        entry->mode = MODE_DIR;
        entry->hash = child_id;
        snprintf(entry->name, sizeof(entry-
>name), "%s", dirname);

        i = end;
    }

    void *serialized = NULL;
    size_t serialized_len = 0;
    if (tree_serialize(&tree, &serialized,
&serialized_len) != 0) return -1;

    int rc = object_write(OBJ_TREE,
serialized, serialized_len, id_out);
    free(serialized);
    return rc;
}

static int tree_index_load(Index *index) {
    index->count = 0;
    FILE *f = fopen(INDEX_FILE, "r");
```

```
    if (!f) return 0;

    while (index->count <
MAX_INDEX_ENTRIES) {
        IndexEntry *e = &index-
>entries[index->count];
        char hex[HASH_HEX_SIZE + 1];

        if (fscanf(f, "%o %64s %llu %u
%511s\n",
                    &e->mode,
                    hex,
                    (unsigned long long *)&e-
>mtime_sec,
                    &e->size,
                    e->path) != 5) {
            break;
        }

        if (hex_to_hash(hex, &e->hash) != 0)
{
            fclose(f);
```

```
            return -1;
        }
        index->count++;
    }

    fclose(f);
    return 0;
}

int tree_from_index(ObjectID *id_out) {
    Index index;

    if (tree_index_load(&index) != 0)
        return -1;

    return
write_tree_recursive(index.entries,
index.count, 0, id_out);
}
```

c) index.c

```
// index.c — Staging area implementation
//
// Text format of .pes/index (one entry per line, sorted by path):
//
//   <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
//
// Example:
//   100644 a1b2c3d4e5f6...  1699900000 42 README.md
//   100644 f7e8d9c0b1a2...  1699900100 128 src/main.c
//
// This is intentionally a simple text format. No magic numbers, no
// binary parsing. The focus is on the staging area CONCEPT (tracking
```

```
// what will go into the next commit) and
ATOMIC WRITES (temp+rename).
//
// PROVIDED functions: index_find,
index_remove, index_status
// TODO functions:     index_load,
index_save, index_add

#include "index.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>

#define MODE_FILE 0100644
#define MODE_EXEC 0100755

// Forward declaration (implemented in
object.c)
```

```
int object_write(ObjectType type, const
void *data, size_t len, ObjectID *id_out);

// ——— PROVIDED
————————————————————————————————————————————————————————————————————————————

// Find an index entry by path (linear
scan).
IndexEntry* index_find(Index *index,
const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path,
path) == 0)
            return &index->entries[i];
    }
    return NULL;
}

// Remove a file from the index.
```

```
// Returns 0 on success, -1 if path not in
index.
int index_remove(Index *index, const char
*path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path,
path) == 0) {
            int remaining = index->count - i -
1;
            if (remaining > 0)
                memmove(&index->entries[i],
&index->entries[i + 1],
                        remaining *
sizeof(IndexEntry));
            index->count--;
            return index_save(index);
        }
    }
    fprintf(stderr, "error: '%s' is not in the
index\n", path);
    return -1;
}
```

```
// Print the status of the working directory.
//
// Identifies files that are staged, unstaged
(modified/deleted in working dir),
// and untracked (present in working dir
but not in index).
// Returns 0.
int index_status(const Index *index) {
    printf("Staged changes:\n");
    int staged_count = 0;
    // Note: A true Git implementation
deeply diffs against the HEAD tree here.
    // For this lab, displaying indexed files
represents the staging intent.
    for (int i = 0; i < index->count; i++) {
        printf("  staged:     %s\n", index-
>entries[i].path);
        staged_count++;
    }
    if (staged_count == 0) printf("  (nothing
to show)\n");
```

```
    printf("\n");

    printf("Unstaged changes:\n");
    int unstaged_count = 0;
    for (int i = 0; i < index->count; i++) {
        struct stat st;
        if (stat(index->entries[i].path, &st) !=
0) {
            printf("  deleted:    %s\n", index-
>entries[i].path);
            unstaged_count++;
        } else {
            // Fast diff: check metadata instead
of re-hashing file content
            if (st.st_mtime != (time_t)index-
>entries[i].mtime_sec || st.st_size !=
(off_t)index->entries[i].size) {
                printf("  modified:   %s\n",
index->entries[i].path);
                unstaged_count++;
            }
        }
```

```
    }
    if (unstaged_count == 0) printf("
(nothing to show)\n");
    printf("\n");

    printf("Untracked files:\n");
    int untracked_count = 0;
    DIR *dir = opendir(".");
    if (dir) {
        struct dirent *ent;
        while ((ent = readdir(dir)) != NULL)
{
            // Skip hidden directories, parent
directories, and build artifacts
            if (strcmp(ent->d_name, ".") == 0 ||
strcmp(ent->d_name, "..") == 0) continue;
            if (strcmp(ent->d_name, ".pes") ==
0) continue;
            if (strcmp(ent->d_name, "pes") ==
0) continue; // compiled executable
            if (strstr(ent->d_name, ".o") !=
NULL) continue; // object files
```

```
        // Check if file is tracked in the
index
        int is_tracked = 0;
        for (int i = 0; i < index->count; i+
+) {
            if (strcmp(index->entries[i].path,
ent->d_name) == 0) {
                is_tracked = 1;
                break;
            }
        }

        if (!is_tracked) {
            struct stat st;
            stat(ent->d_name, &st);
            if (S_ISREG(st.st_mode)) { //
Only list regular files for simplicity
                printf("  untracked:  %s\n",
ent->d_name);
                untracked_count++;
            }
```

```
            }
        }
        closedir(dir);
    }
    if (untracked_count == 0) printf("
(nothing to show)\n");
    printf("\n");

    return 0;
}

// ——— TODO: Implement these
———————————————
```

```
    return strcmp(ea->path, eb->path);
}

int index_load(Index *index) {
    index->count = 0;
    FILE *f = fopen(INDEX_FILE, "r");
    if (!f) return 0; // Empty index is
acceptable

    while (index->count <
MAX_INDEX_ENTRIES) {
        IndexEntry *e = &index-
>entries[index->count];
        char hex[HASH_HEX_SIZE + 1];
        // Format: <mode> <hash> <mtime>
<size> <path>
        if (fscanf(f, "%o %64s %llu %u
%511s\n",
            &e->mode, hex, (unsigned long
long *)&e->mtime_sec, &e->size, e-
>path) != 5) break;
```

```
        hex_to_hash(hex, &e->hash);
        index->count++;
    }
    fclose(f);
    return 0;
}

int index_save(const Index *index) {
    char tmp_path[] = INDEX_FILE
".tmp";
    FILE *f = fopen(tmp_path, "w");
    if (!f) return -1;

    IndexEntry *sorted_entries =
malloc((size_t)index->count *
sizeof(IndexEntry));
    if (!sorted_entries) {
        fclose(f);
        return -1;
    }
```

```
    memcpy(sorted_entries, index->entries,
(size_t)index->count *
sizeof(IndexEntry));
    qsort(sorted_entries, index->count,
sizeof(IndexEntry),
compare_index_entries);

    for (int i = 0; i < index->count; i++) {
        const IndexEntry *e =
&sorted_entries[i];
        char hex[HASH_HEX_SIZE + 1];
        hash_to_hex(&e->hash, hex);
        fprintf(f, "%o %s %llu %u %s\n",
                e->mode, hex, (unsigned long
long)e->mtime_sec, e->size, e->path);
    }

    fflush(f);
    fsync(fileno(f));
    fclose(f);
    int rc = rename(tmp_path,
INDEX_FILE);
```

```
    free(sorted_entries);
    return rc;
}

int index_add(Index *index, const char
*path) {
    struct stat st;
    if (lstat(path, &st) != 0) return -1;
    if (!S_ISREG(st.st_mode)) return -1;

    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    size_t file_size = (size_t)st.st_size;
    void *buf = malloc(file_size ?
file_size : 1);
    if (!buf) {
        fclose(f);
        return -1;
    }
    if (fread(buf, 1, file_size, f) != file_size)
{
```

```
        free(buf); fclose(f); return -1;
    }
    fclose(f);

    ObjectID blob_id;
    if (object_write(OBJ_BLOB, buf,
file_size, &blob_id) != 0) {
        free(buf); return -1;
    }
    free(buf);

    IndexEntry *entry = index_find(index,
path);
    if (!entry) {
        if (index->count >=
MAX_INDEX_ENTRIES) return -1;
        entry = &index->entries[index-
>count++];
        strncpy(entry->path, path,
sizeof(entry->path) - 1);
        entry->path[sizeof(entry->path) - 1]
= '\0';
```

```
    }

    entry->mode = (st.st_mode &
S_IXUSR) ? MODE_EXEC :
MODE_FILE;
    entry->hash = blob_id;
    entry->mtime_sec =
(uint64_t)st.st_mtime;
    entry->size = (uint32_t)st.st_size;

    return index_save(index);
}
```

d) commit.c

```
// commit.c — Commit creation and
history traversal
//
// Commit object format (stored as text,
one field per line):
//
```

```
//   tree <64-char-hex-hash>
//   parent <64-char-hex-hash>        ←
omitted for the first commit
//   author <name> <unix-timestamp>
//   committer <name> <unix-timestamp>
//
//   <commit message>
//
// Note: there is a blank line between the
headers and the message.
//
// PROVIDED functions: commit_parse,
commit_serialize, commit_walk,
head_read, head_update
// TODO functions:     commit_create

#include "commit.h"
#include "index.h"
#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <inttypes.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>

// Forward declarations (implemented in
object.c)
int object_write(ObjectType type, const
void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id,
ObjectType *type_out, void **data_out,
size_t *len_out);

// ——— PROVIDED
————————————
```

```
    (void)len;
    const char *p = (const char *)data;
    char hex[HASH_HEX_SIZE + 1];

    // "tree <hex>\n"
    if (sscanf(p, "tree %64s\n", hex) != 1)
return -1;
    if (hex_to_hash(hex, &commit_out-
>tree) != 0) return -1;
    p = strchr(p, '\n') + 1;

    // optional "parent <hex>\n"
    if (strncmp(p, "parent ", 7) == 0) {
        if (sscanf(p, "parent %64s\n", hex) !=
1) return -1;
        if (hex_to_hash(hex, &commit_out-
>parent) != 0) return -1;
        commit_out->has_parent = 1;
        p = strchr(p, '\n') + 1;
    } else {
        commit_out->has_parent = 0;
    }
```

```
    // "author <name> <timestamp>\n"
    char author_buf[256];
    uint64_t ts;
    if (sscanf(p, "author %255[^\n]\n",
author_buf) != 1) return -1;
    // split off trailing timestamp
    char *last_space = strrchr(author_buf, '
');
    if (!last_space) return -1;
    ts = (uint64_t)strtoull(last_space + 1,
NULL, 10);
    *last_space = '\0';
    snprintf(commit_out->author,
sizeof(commit_out->author), "%s",
author_buf);
    commit_out->timestamp = ts;
    p = strchr(p, '\n') + 1;  // skip author line
    p = strchr(p, '\n') + 1;  // skip committer
line
    p = strchr(p, '\n') + 1;  // skip blank line
```

```
    snprintf(commit_out->message,
sizeof(commit_out->message), "%s", p);
    return 0;
}

// Serialize a Commit struct to the text
format.
// Caller must free(*data_out).
int commit_serialize(const Commit
*commit, void **data_out, size_t
*len_out) {
    char tree_hex[HASH_HEX_SIZE + 1];
    char parent_hex[HASH_HEX_SIZE +
1];
    hash_to_hex(&commit->tree, tree_hex);

    char buf[8192];
    int n = 0;
    n += snprintf(buf + n, sizeof(buf) - n,
"tree %s\n", tree_hex);
    if (commit->has_parent) {
```

```
        hash_to_hex(&commit->parent,
parent_hex);
        n += snprintf(buf + n, sizeof(buf) - n,
"parent %s\n", parent_hex);
    }
    n += snprintf(buf + n, sizeof(buf) - n,
                "author %s %" PRIu64 "\n"
                "committer %s %" PRIu64 "\n"
                "\n"
                "%s",
                commit->author, commit-
>timestamp,
                commit->author, commit-
>timestamp,
                commit->message);

    *data_out = malloc(n + 1);
    if (!*data_out) return -1;
    memcpy(*data_out, buf, n + 1);
    *len_out = (size_t)n;
    return 0;
}
```

```
// Walk commit history from HEAD to the
root.
int commit_walk(commit_walk_fn
callback, void *ctx) {
    ObjectID id;
    if (head_read(&id) != 0) return -1;

    while (1) {
        ObjectType type;
        void *raw;
        size_t raw_len;
        if (object_read(&id, &type, &raw,
&raw_len) != 0) return -1;

        Commit c;
        int rc = commit_parse(raw, raw_len,
&c);
        free(raw);
        if (rc != 0) return -1;

        callback(&id, &c, ctx);
```

```
        if (!c.has_parent) break;
        id = c.parent;
    }
    return 0;
}

// Read the current HEAD commit hash.
int head_read(ObjectID *id_out) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f))
{ fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0'; // strip
newline

    char ref_path[512];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(ref_path, sizeof(ref_path),
"%s/%s", PES_DIR, line + 5);
```

```
        f = fopen(ref_path, "r");
        if (!f) return -1; // Branch exists but
has no commits yet
        if (!fgets(line, sizeof(line), f))
{ fclose(f); return -1; }
        fclose(f);
        line[strcspn(line, "\r\n")] = '\0';
    }
    return hex_to_hash(line, id_out);
}

// Update the current branch ref to point to
a new commit atomically.
int head_update(const ObjectID
*new_commit) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f))
{ fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0';
```

```
    char target_path[520];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(target_path,
sizeof(target_path), "%s/%s", PES_DIR,
line + 5);
    } else {
        snprintf(target_path,
sizeof(target_path), "%s",
HEAD_FILE); // Detached HEAD
    }

    char tmp_path[528];
    snprintf(tmp_path, sizeof(tmp_path),
"%s.tmp", target_path);

    f = fopen(tmp_path, "w");
    if (!f) return -1;

    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(new_commit, hex);
    fprintf(f, "%s\n", hex);
```

```
    fflush(f);
    fsync(fileno(f));
    fclose(f);

    return rename(tmp_path, target_path);
}

// ——— TODO: Implement these ————————————————————————————————————————————————————

// Create a new commit from the current staging area.
//
// HINTS - Useful functions to call:
//   - tree_from_index   : writes the directory tree and gets the root hash
//   - head_read         : gets the parent commit hash (if any)
```

```
//   - pes_author        : retrieves the author
name string (from pes.h)
//   - time(NULL)        : gets the current
unix timestamp
//   - commit_serialize  : converts the filled
Commit struct to a text buffer
//   - object_write      : saves the serialized
text as OBJ_COMMIT
//   - head_update       : moves the branch
pointer to your new commit
//
// Returns 0 on success, -1 on error.
int commit_create(const char *message,
ObjectID *commit_id_out) {
    Commit commit;
    memset(&commit, 0, sizeof(commit));

    if (tree_from_index(&commit.tree) !=
0) return -1;

    if (head_read(&commit.parent) == 0) {
        commit.has_parent = 1;
```

```
    } else {
        commit.has_parent = 0;
    }

    snprintf(commit.author,
sizeof(commit.author), "%s",
pes_author());
    commit.timestamp =
(uint64_t)time(NULL);
    snprintf(commit.message,
sizeof(commit.message), "%s", message ?
message : "");

    void *raw = NULL;
    size_t raw_len = 0;
    if (commit_serialize(&commit, &raw,
&raw_len) != 0) return -1;

    int rc = object_write(OBJ_COMMIT,
raw, raw_len, commit_id_out);
    free(raw);
    if (rc != 0) return -1;
```

```
    if (head_update(commit_id_out) != 0)
return -1;
    return 0;
}
```

8. Github Repository link: -

https://github.com/EnforcerXII/PES2UG24CS920-pes-vcs